

Normalización de datos con PostgreSQL — Clientes y Pedidos

Laboratorio práctico de migración de una planilla de Excel no normalizada hacia un esquema relacional en PostgreSQL: diseño de tablas, integridad referencial, limpieza de datos y resolución de errores reales durante el proceso.

Mi proyecto consta de

- 1 Terminar de normalizar y migrar los datos de clientes/pedidos.
- 2 Diseñar y crear las 5 tablas de costos (insumos, productos, despiece, proveedores, insumo_proveedor), y migrar los datos reales del Excel de costos.
- 3 Recién con la base de datos completa y poblada, armar el programa en C# que consuma esa base.

Paso 1 — Base de datos y tablas principales

Primero creé mi base de datos, luego creé mis tablas:

```
CREATE TABLE clientes (  
  id_cliente SERIAL PRIMARY KEY,  
  nombre TEXT,  
  telefono VARCHAR(20),  
  medio_contacto VARCHAR(30),  
  provincia_ciudad VARCHAR(50),  
  es_revendedor BOOLEAN  
);  
  
CREATE TABLE pedidos (  
  id_pedido SERIAL PRIMARY KEY,  
  id_cliente INT REFERENCES clientes(id_cliente),  
  producto VARCHAR(50),  
  modelo VARCHAR(50),  
  talla VARCHAR(5),  
  estado VARCHAR(20),  
  fecha_pedido DATE,  
  fecha_entrega DATE,  
  forma_pago VARCHAR(30),  
  medio_envio VARCHAR(30),  
  canal_venta VARCHAR(30),  
  usuario_plataforma VARCHAR(50),  
  valor_unitario NUMERIC(10,2),  
  cantidad INT,  
  total NUMERIC(10,2) GENERATED ALWAYS AS (valor_unitario * cantidad) STORED  
);
```

Paso 2 — Prueba de la estructura

Luego inserté valores de prueba para chequear que todo esté bien configurado.

Query History

```

1 SELECT * FROM pedidos
2
3 INSERT into pedidos (id_pedido, id_cliente, producto, modelo, talla, estado, fecha_pedido, fecha_entrega, forma_pago, medio_envio, valor
4 VALUES (1, 5, 'Remera', 'colapinto', 'M', 'Pendiente', CURRENT_DATE, CURRENT_DATE, 'efectivo', 'correo', 10, 3
5 );
6
7

```

Data Output Messages Notifications

ERROR: inserción o actualización en la tabla «pedidos» viola la llave foránea «pedidos_id_cliente_fkey»
La llave (id_cliente)=(5) no está presente en la tabla «clientes».

SQL state: 23503
Detail: La llave (id_cliente)=(5) no está presente en la tabla «clientes».

ERROR: inserción o actualización en la tabla «pedidos» viola la llave foránea «pedidos_id_cliente_fkey».
La llave (id_cliente)=(5) no está presente en la tabla «clientes».

Corregí el id_cliente por uno existente en la tabla clientes y la inserción funcionó correctamente:

	ig (20)	fecha_pedido date	fecha_entrega date	forma_pago character varying (30)	medio_envio character varying (30)	canal_venta character varying (30)	usuario_plataforma character varying (50)	valor_unitario numeric (10,2)	cantidad Integer	total numeric (10,2)
1		2026-09-12	2026-09-12	efectivo	correo	[null]	[null]	10.00	3	30.00

Paso 3 — Tabla temporal (staging) para importar el Excel

Luego creé la tabla “descartable” para traer la información del Excel. Para esto, pasé primero las primeras 30 filas del Excel a otro archivo nuevo y lo exporté en formato CSV.

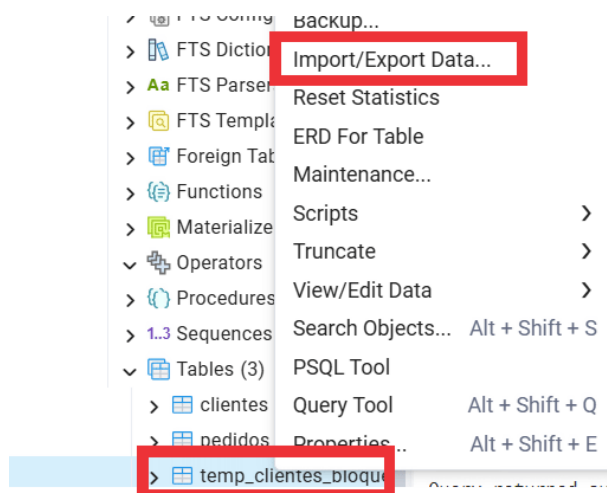
```

CREATE TABLE temp_clientes_bloque1 (
  nombre_cliente VARCHAR(100),
  comentario VARCHAR(100),
  enviado VARCHAR(50),
  usuario VARCHAR(100),
  telefono VARCHAR(50),
  talla VARCHAR(20),
  consulta_por VARCHAR(150),
  vendedor VARCHAR(50),
  envio VARCHAR(100)
);

```

Como muchos de los datos estaban inexactos e inestables, puse todas las variables como VARCHAR.

Luego de crear la tabla, importé el Excel:



Paso 4 — Limpieza de datos

Analicé la tabla a importar y, debido a que hay varios campos sin información, utilicé SQL para completar estos campos vacíos. Por ejemplo, veo que hay muchos usuarios sin el nombre y nombres sin usuario, así que los acomodé con UPDATE:

```
UPDATE tabla
SET columna = nuevo_valor
WHERE condición;

UPDATE temp_clientes_bloque1
SET nombre_cliente = usuario
WHERE nombre_cliente IS NULL;
```

Paso 5 — Primera migración a clientes

Una vez acomodados todos los datos, paso a importar las tablas.

```
INSERT INTO clientes (nombre, telefono, medio_contacto, provincia_ciudad,
es_revendedor)
SELECT DISTINCT nombre_cliente, telefono, NULL, NULL, FALSE
FROM temp_clientes_bloque1;
```

Evalúo las inconsistencias y pienso, según el contexto, que me conviene tomar de parámetro para evitar la duplicidad de ID de cliente. Siempre se toman riesgos en base a esto. Para eso es necesario conocer de qué trata el negocio y la planilla que se está resolviendo, y para qué nos va a servir a futuro.

Teniendo en cuenta que Postgres empareja por posición, no por nombre — es decir, no mira si se llama “nombre_cliente” o “nombre”, sino que la primera columna del INSERT recibe lo que devuelve la primera columna del SELECT, la segunda con la segunda, y así sucesivamente, en orden estricto — ejecuto el comando con cuidado de no cruzar los datos.

ME SALTÓ UN ERROR

```
ERROR: el valor es demasiado largo para el tipo character varying(20)
SQL state: 22001
```

Vamos a ver si hay algún valor que supere los 20 caracteres:

```
SELECT telefono, LENGTH(telefono) AS largo
FROM temp_clientes_bloque1
ORDER BY largo DESC;
```

Verifico que un teléfono tiene una nota al lado, así que:

```
UPDATE temp_clientes_bloque1
SET telefono = 'nuevo_valor'
WHERE telefono = 'valor_viejo_exacto';
```

PROBLEMA RESUELTO

Paso 6 — Relacionar con JOIN

Ahora, para pasar todos los datos a la otra tabla, me encuentro con que si paso el id_cliente (que es una FK) y no se encuentra en la tabla original (temp_clientes_bloque1), este tendría que asignarle una FK nueva, y eso no tiene que suceder. Para esto usamos la función JOIN, que me relaciona el dato id_cliente de la tabla clientes con la columna nombre de la tabla temp_clientes_bloque1, para asociarlo con la FK correspondiente.

DILEMA

Acá tengo en la tabla pedidos un campo para producto y otro para modelo, que en la tabla temporal temp_clientes_bloque1 tengo ambos datos en una sola columna, que es consulta_por. Acá uno tiene que analizar según la necesidad del cliente y del negocio qué decisión tomamos. En este caso decidí duplicar las columnas y luego, en todo caso, realizar la modificación correspondiente. Entonces:

```
producto → consulta_por
modelo → consulta_por
```

DILEMA RESUELTO

Entonces:

```
INSERT INTO pedidos (id_cliente, producto, modelo, talla, usuario_plataforma,
estado,
fecha_pedido, fecha_entrega, forma_pago, medio_envio, canal_venta,
valor_unitario, cantidad)
SELECT clientes.id_cliente, consulta_por, consulta_por, talla, usuario,
NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL
FROM temp_clientes_bloque1
JOIN clientes ON clientes.nombre = temp_clientes_bloque1.nombre_cliente;
```

¿Hasta aquí qué hice?

- Diseño relacional: separación de datos fijos (Clientes) vs. variables (Pedidos), aplicando la regla “¿puede cambiar entre pedidos del mismo cliente?”
- Tipos de dato: VARCHAR vs TEXT, NUMERIC vs FLOAT (por qué NUMERIC para dinero), BOOLEAN, SERIAL, DATE.
- Restricciones: PRIMARY KEY, FOREIGN KEY con REFERENCES, columna calculada con GENERATED ALWAYS AS ... STORED.
- Tabla de staging (temporal) como buffer de datos crudos antes de normalizar.

Ya ejecutado esto, veo con:

```
SELECT DISTINCT nombre_cliente
FROM temp_clientes_bloque1
WHERE nombre_cliente IN (SELECT nombre FROM clientes);
```

los nombres que ya estaban y no había que agregar. Y con:

```
SELECT DISTINCT nombre_cliente
FROM temp_clientes_bloque1
WHERE nombre_cliente NOT IN (SELECT nombre FROM clientes);
```

los que no estaban y se deben agregar. Los resultados me dan que:

- En mi tabla original seleccioné 29 filas.
- El primer comando (IN) me da 22 coincidencias.
- El segundo comando (NOT IN) me da cero.

Ahora debería ver, en mi tabla de descarte, cuántos nombres se repitieron ahí mismo, porque capaz que en vez de 29 terminan siendo 22, y ahí me cierra el número. Para comprobar esto con SQL en vez de asumir:

```
SELECT nombre_cliente, COUNT(*) AS veces
FROM temp_clientes_bloque1
GROUP BY nombre_cliente
ORDER BY veces DESC;
```

Esto me agrupa cada nombre y cuenta cuántas veces aparece en el bloque completo (29 filas totales, sin DISTINCT), para ver si los 7 que me faltan son 7 repetidos. Con esto verifiqué que efectivamente 22 eran las filas reales y 7 eran repetidos, así que este paso ya está cumplido. Por ende, si primero hubiera chequeado las coincidencias entre tablas, me ahorra el insert.

Conclusión y aprendizaje para la próxima — invertir el orden (chequear coincidencias antes del INSERT) me ahorra ejecutar algo que ya sé que no va a insertar nada. Anotado como mejora de proceso para el próximo bloque.

Migración del segundo bloque — Tabla Pedidos

Acá se pueden repetir los usuarios siempre y cuando no se me agregue un nuevo id de cliente: ese debe ser el mismo que en la tabla clientes, que en la tabla temporal no está. Así que no hace falta que se fije que no aparezca el mismo usuario, porque en efecto, al ser un pedido diferente, va a repetirse. Así que no se usa WHERE NOT IN.

Así que:

```
INSERT INTO pedidos (id_cliente, producto, modelo, talle, usuario_plataforma,
estado,
fecha_pedido, fecha_entrega, forma_pago, medio_envio, canal_venta,
valor_unitario, cantidad)
SELECT clientes.id_cliente, consulta_por, consulta_por, talle, usuario,
NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL
FROM temp_clientes_bloque1
JOIN clientes ON clientes.nombre = temp_clientes_bloque1.nombre_cliente;
```

Checklist antes de seguir con los próximos bloques

- 1 Exportar el bloque a CSV desde Excel.
- 2 Vaciar o recrear temp_clientes_bloque1.
- 3 Importar el CSV (Import/Export en pgAdmin, con Header activado).
- 4 UPDATE para completar nombre_cliente con usuario donde esté NULL.
- 5 UPDATE para completar telefono con S/D donde esté NULL.
- 6 Revisar con SELECT * si hay casos raros (texto pegado, etc.).
- 7 Chequear coincidencias con clientes antes de insertar (SELECT ... WHERE nombre_cliente IN (SELECT nombre FROM clientes)).
- 8 INSERT INTO clientes ... WHERE nombre_cliente NOT IN (...).
- 9 INSERT INTO pedidos ... JOIN clientes ON ...

Historial de comandos de este segundo proceso

```

SELECT * FROM public.temp_clientes_bloque1;

UPDATE temp_clientes_bloque1
SET nombre_cliente = usuario
WHERE nombre_cliente IS NULL;

UPDATE temp_clientes_bloque1
SET telefono = 'S/D'
WHERE telefono IS NULL;

UPDATE temp_clientes_bloque1
SET enviado = 'avisado'
WHERE enviado IS NULL;

UPDATE temp_clientes_bloque1
SET telefono = 'S/D'
WHERE telefono = '10%';

INSERT INTO clientes (nombre, telefono, medio_contacto, provincia_ciudad,
es_revendedor)
SELECT DISTINCT nombre_cliente, telefono, NULL, NULL, FALSE
FROM temp_clientes_bloque1
WHERE nombre_cliente NOT IN (SELECT nombre FROM clientes);

SELECT DISTINCT nombre_cliente
FROM temp_clientes_bloque1
WHERE nombre_cliente IN (SELECT nombre FROM clientes);

SELECT DISTINCT nombre_cliente
FROM temp_clientes_bloque1
WHERE nombre_cliente NOT IN (SELECT nombre FROM clientes);

SELECT nombre_cliente, COUNT(*) AS veces
FROM temp_clientes_bloque1
GROUP BY nombre_cliente
ORDER BY veces DESC;

INSERT INTO pedidos (id_cliente, producto, modelo, talla, usuario_plataforma,
estado,
fecha_pedido, fecha_entrega, forma_pago, medio_envio, canal_venta,
valor_unitario, cantidad)
SELECT clientes.id_cliente, consulta_por, consulta_por, talla, usuario,
NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL
FROM temp_clientes_bloque1
JOIN clientes ON clientes.nombre = temp_clientes_bloque1.nombre_cliente;

```